



Progressive Education Society's  
**MODERN COLLEGE OF ENGINEERING, Pune -05.**  
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)  
**MCA Department**

**PRACTICAL SUBMISSION RECORD- A.Y. 2025-26**

|                                             |                |                                                                             |                  |
|---------------------------------------------|----------------|-----------------------------------------------------------------------------|------------------|
| <b>Class: SYMCA</b><br><b>Semester: III</b> | <b>Div.: B</b> | <b>Course Code: MCA01604</b><br><b>Course Name: Data Science Laboratory</b> | <b>Batch: S2</b> |
| <b>Name: Ajit Bajarang Mali</b>             |                | <b>Roll No: 52140</b>                                                       |                  |
| <b>CO No: CO605.1</b>                       |                | <b>Assignment No: 1</b>                                                     |                  |

**Title:** Use the Naive Bayes classifier to perform sentiment analysis on a dataset of movie reviews, classifying them as either positive or negative based on the review text.

**Code:-**

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, classification_report

# 1. Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
target_names = iris.target_names

# 2. Prepare the data for binary classification
# We want to predict 'Iris-setosa' (class 0) vs. 'not Iris-setosa' (classes 1 and 2).
# Let's make 'Iris-setosa' our positive class (1) and the others the negative class (0).
y_binary = (y == 0).astype(int) # 1 if Iris-setosa (y==0), else 0

# 3. Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y_binary, test_size=0.3, random_state=42, stratify=y_binary
)

# 4. Train the Logistic Regression model
model = LogisticRegression(random_state=42)
model.fit(X_train, y_train)

# 5. Make predictions on the test set
y_pred = model.predict(X_test)

# 6. Evaluate the model's performance
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
report = classification_report(y_test, y_pred, target_names=['Not Setosa', 'Is Setosa'])
```



```
# 7. Display the results
print("--- Model Evaluation for Iris-setosa Classification ---")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print("\n--- Detailed Classification Report ---")
print(report)
```

**Output:-**

```
--- Model Evaluation for Iris-setosa Classification ---
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000

--- Detailed Classification Report ---
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Not Setosa   | 1.00      | 1.00   | 1.00     | 30      |
| Is Setosa    | 1.00      | 1.00   | 1.00     | 15      |
| accuracy     |           |        | 1.00     | 45      |
| macro avg    | 1.00      | 1.00   | 1.00     | 45      |
| weighted avg | 1.00      | 1.00   | 1.00     | 45      |